

Motif Finding

BioInf 2025/2026 1st Semester

André Lamúrias

a.lamurias@fct.unl.pt

Recap

- Pairwise Sequence Alignment: insert gaps to determine what two sequences have in common
- Multiple Sequence Alignments: find what multiple sequences have in common
- Database search: find matches with heuristics
- Phylogenetic Trees: study the evolutionary relationships between sequences

Transcription Factors

- How does the cell know it should produce some protein?
- A cell has many genes, but they are not all expressed at the same time
 - That would be a waste of resources
- Example: Plants produce some proteins during night and other during the day
- Gene expression is regulated by **Transcription Factors (TFs)**
 - TFs are proteins that bind to certain parts of the DNA, controlling its transcription

Transcription Factors

- TFs bind to **regulatory motifs** aka **transcription factor binding sites**
- Knowing the sequence of these regulatory motifs could improve our knowledge of the inner working of a cell
- But this is not trivial: TFs are not completely conserved i.e. there are always variations
 - Example: CCA1 can bind to AAAAAATCT and AAG**G**AA**C**TCT

Motif Finding

- Example of NF-κB TF binding sites from the *Drosophila melanogaster* genome

1	T	C	G	G	G	G	g	T	T	T	t	t
2	c	C	G	G	t	G	A	c	T	T	a	C
3	a	C	G	G	G	G	A	T	T	T	t	C
4	T	t	G	G	G	G	A	c	T	T	t	t
5	a	a	G	G	G	G	A	c	T	T	C	C
6	T	t	G	G	G	G	A	c	T	T	C	C
7	T	C	G	G	G	G	A	T	T	c	a	t
8	T	C	G	G	G	G	A	T	T	c	C	t
9	T	a	G	G	G	G	A	a	c	T	a	C
10	T	C	G	G	G	t	A	T	a	a	C	C

Motif Finding

- Find the hidden 15-mer in these 10 sequences

```
1  atgaccgggatactgatAAAAAAAGGGGGGggcgtaacacattagataaacgtatgaagtacgttagactcggcgccgccg
2  acccctatTTTTTgagcagatttagtgacctggaaaaaaatttgagtacaaaactTTTccgaataAAAAAAAGGGGGGa
3  tgagtatccctgggatgacttAAAAAAAGGGGGGtgctctcccgattTTTgaatatgtaggatcattcgccaggggtccga
4  gctgagaattggatgAAAAAAAGGGGGGtccacgcaatcgcgaaaccaacgcggacccaaaggcaagaccgataaaggaga
5  tccctTTTgcggtaatgtgccgggaggctggttacgtaggggaagccctaacggacttaatAAAAAAAGGGGGGcttatag
6  gtcaatcatgttcttgtgaatggatttAAAAAAAGGGGGGgaccgcttggcgcacccaaattcagtggtgggcgagcgcaa
7  cggTTTTggcccttgtttagaggcccccgTAAAAAAAGGGGGGcaattatgagagagctaattctatcgcgTgcgtgttcat
8  aacttgagttAAAAAAAGGGGGGctggggcacatacaagaggagtcttccttatcagttaatgctgtatgacactatgta
9  ttggcccatTggctaaaagcccaacttgacaaatggaagatagaatccttgcatAAAAAAAGGGGGGaccgaaagggaag
10 ctggtgagcaacgacagattcttacgtgcattagctcgcttccgggatctaatagcacgaagcttAAAAAAAGGGGGGa
```

- Solution: Concatenate and apply algorithms from Lecture 4 (Frequent Word Problem)

Motif Finding

- What if the 15-mer pattern is not exactly the same, with 4 altered position in each sequence?

```
1  atgaccgggatactgatAgAAGAAAGGttGGGggcgtacacattagataaacgtatgaagtacgttagactcggcgccgcccg
2  acccctatTTTTTgagcagatttagtgacctggaaaaaaaaatttgagtacaaaacttttccgaatacAAATAAAcGGcGGGa
3  tgagtatccctgggatgacttAAAAtAATGGaGtGGtgctctcccgatttttgaatatgtaggatcattcgccaggggtccga
4  gctgagaattggatgcAAAAAAGGGattGtccacgcaatcgcgaaaccaacgcggacccaaagggaagaccgataaaggaga
5  tcccttttgcggtaatgtgccgggaggctggttacgtagggaagccctaacggacttaatAtAATAAAGGaaGGGccttatag
6  gtcaatcatgttcttgtgaatggatttAAcAATAAGGGctGGgaccgcttggcgcacccaaattcagtgtgggcgagcgcaa
7  cggttttggcccttgtagaggcccccgAtAAAcAAGGaGGGccaattatgagagagctaatactatcgcggtgcgtgttcat
8  aacttgagttAAAAAAtAGGGaGccctggggcacatacaagaggagtcttccttatcagttaatgctgtatgacactatgta
9  ttggccattggctaaaagcccaacttgacaaatggaagatagaatccttgcatActAAAAAGGaGcGGaccgaaagggaag
10 ctggtgagcaacgacagattcttacgtgcattagctcgcttccggggatctaatagcacgaagcttActAAAAAGGaGcGGa
```

- We have considered mismatches before but it is not going to scale
- Also, concatenating different DNA string does not make sense
- Distinct from MSA

Motif Finding

- (k,d) -motif: A k -mer that appears in every string with at most d mismatches
-

Implanted Motif Problem:

Find all (k,d) -motifs in a collection of strings.

Input: A collection of strings Dna , and integers k and d .

Output: All (k,d) -motifs in Dna .

Brute force search (Lab7 BA2A)

```
MOTIFENUMERATION(Dna, k, d)  
  Patterns  $\leftarrow$  an empty set  
  for each k-mer Pattern in Dna  
    for each k-mer Pattern' differing from Pattern by at most d mismatches  
      if Pattern' appears in each string from Dna with at most d mismatches  
        add Pattern' to Patterns  
  remove duplicates from Patterns  
  return Patterns
```

- *Pattern* occurs in *Dna* but *Pattern'* does not have to occur
- How to make this faster?

Brute force search

- Identify the two most similar k-mers between each pair of string
 - However two k-mers might be more different from each other than from the correct motif
 - Consider:
 - AAAAAAAAAAGGGGGGGG (correct motif)
 - AgAAgAAAGGttGGG (4 mismatches)
 - cAAtAAAACGGGGcG (4 mismatches)
 - 8 mismatches between the last two kmers
- Pairwise comparison is not going to work

Scoring Motifs

- We can score individual instances of a motif (given by a position in a sequence corresponding to a k -mer) by comparing with an “ideal” motif
- We do not know this “ideal” motif
- We score depending on how similar their k -mers are to each other
- Consider
 - t DNA strings
 - Each string with length n
- Select one k -mer from each string to form the collection *Motifs* ($t \times k$)
- We can calculate the score of *Motifs* based how similar each column is

Scoring Motifs

Motifs

T	C	G	G	G	G	g	T	T	T	t	t
c	C	G	G	t	G	A	c	T	T	a	C
a	C	G	G	G	G	A	T	T	T	t	C
T	t	G	G	G	G	A	c	T	T	t	t
a	a	G	G	G	G	A	c	T	T	C	C
T	t	G	G	G	G	A	c	T	T	C	C
T	C	G	G	G	G	A	T	T	c	a	t
T	C	G	G	G	G	A	T	T	c	C	t
T	a	G	G	G	G	A	a	c	T	a	C
T	C	G	G	G	t	A	T	a	a	C	C

SCORE(*Motifs*)

$$3 + 4 + 0 + 0 + 1 + 1 + 1 + 5 + 2 + 3 + 6 + 4 = 30$$

Profile Matrix

Motifs		T	C	G	G	G	G	g	T	T	T	t	t
		c	C	G	G	t	G	A	c	T	T	a	C
		a	C	G	G	G	G	A	T	T	T	t	C
		T	t	G	G	G	G	A	c	T	T	t	t
		a	a	G	G	G	G	A	c	T	T	C	C
		T	t	G	G	G	G	A	c	T	T	C	C
		T	C	G	G	G	G	A	T	T	c	a	t
		T	C	G	G	G	G	A	T	T	c	C	t
		T	a	G	G	G	G	A	a	c	T	a	C
		T	C	G	G	G	t	A	T	a	a	C	C
COUNT(Motifs)	A:	2	2	0	0	0	0	9	1	1	1	3	0
	C:	1	6	0	0	0	0	0	4	1	2	4	6
	G:	0	0	10	10	9	9	1	0	0	0	0	0
	T:	7	2	0	0	1	1	0	5	8	7	3	4

Profile Matrix

Motifs	T	C	G	G	G	G	g	T	T	T	t	t	
	c	C	G	G	t	G	A	c	T	T	a	C	
	a	C	G	G	G	G	A	T	T	T	t	C	
	T	t	G	G	G	G	A	c	T	T	t	t	
	a	a	G	G	G	G	A	c	T	T	C	C	
	T	t	G	G	G	G	A	c	T	T	C	C	
	T	C	G	G	G	G	A	T	T	c	a	t	
	T	C	G	G	G	G	A	T	T	c	C	t	
	T	a	G	G	G	G	A	a	c	T	a	C	
T	C	G	G	G	t	A	T	a	a	C	C		
PROFILE(Motifs)	A:	.2	.2	0	0	0	0	.9	.1	.1	.1	.3	0
	C:	.1	.6	0	0	0	0	0	.4	.1	.2	.4	.6
	G:	0	0	1	1	.9	.9	.1	0	0	0	0	0
	T:	.7	.2	0	0	.1	.1	0	.5	.8	.7	.3	.4

Consensus and logo

Motifs

T	C	G	G	G	G	g	T	T	T	t	t
c	C	G	G	t	G	A	c	T	T	a	C
a	C	G	G	G	G	A	T	T	T	t	C
T	t	G	G	G	G	A	c	T	T	t	t
a	a	G	G	G	G	A	c	T	T	C	C
T	t	G	G	G	G	A	c	T	T	C	C
T	C	G	G	G	G	A	T	T	c	a	t
T	C	G	G	G	G	A	T	T	c	C	t
T	a	G	G	G	G	A	a	c	T	a	C
T	C	G	G	G	t	A	T	a	a	C	C
T	C	G	G	G	G	A	T	T	T	C	C

CONSENSUS(*Motifs*)



Better scoring function

- Some positions (columns) have more variance than others
- 2nd and last column both have 4 mismatches
- But last column is either C or T
 - It is “more conserved” so it should receive a lower score

T	C	G	G	G	G	g	T	T	T	t	t
c	C	G	G	t	G	A	c	T	T	a	C
a	C	G	G	G	G	A	T	T	T	t	C
T	t	G	G	G	G	A	c	T	T	t	t
a	a	G	G	G	G	A	c	T	T	C	C
T	t	G	G	G	G	A	c	T	T	C	C
T	C	G	G	G	G	A	T	T	c	a	t
T	C	G	G	G	G	A	T	T	c	C	t
T	a	G	G	G	G	A	a	c	T	a	C
T	C	G	G	G	t	A	T	a	a	C	C

Better scoring function

A:	.2	.2	0	0	0	0	.9	.1	.1	.1	.3	0
C:	.1	.6	0	0	0	0	0	.4	.1	.2	.4	.6
G:	0	0	1	1	.9	.9	.1	0	0	0	0	0
T:	.7	.2	0	0	.1	.1	0	.5	.8	.7	.3	.4

- Each column of the profile matrix sums to 1
- Entropy measures uncertainty of a probability distribution
 - Also used in other contexts (e.g. decision trees, MSA)
 - $H(p_1, \dots, p_N) = -\sum_{i=1}^N p_i \log_2(p_i)$
 - Second column: $-(0.2 \log_2 0.2 + 0.6 \log_2 0.6 + 0.0 \log_2 0.0 + 0.2 \log_2 0.2) \approx 1.371$
 - Last column: $-(0.0 \log_2 0.0 + 0.6 \log_2 0.6 + 0.0 \log_2 0.0 + 0.4 \log_2 0.4) \approx 0.971$
- High entropy – less conserved (just like our previous scoring function)
- The entropy of *Motifs* is the sum of the entropy of each column

Motif Finding

Motif Finding Problem:

Given a collection of strings, find a set of k -mers, one from each string, that minimizes the score of the resulting motif.

Input: A collection of strings Dna and an integer k .

Output: A collection $Motifs$ of k -mers, one from each string in Dna , minimizing $SCORE(Motifs)$ among all possible choices of k -mers.

- The set of k -mers can be represented as the initial position of the that k -mer on each string ($n - k + 1$ choices on each string)
- t is the number of string, n is the length of the strings
- Each choice of $Motifs$ requires $k \cdot t$ steps to score
- Brute force: time complexity of $\mathcal{O}(n^t \cdot k \cdot t)$

Motif Finding

- Instead of the Consensus being determined by the Motifs matrix, what if the matrix was determined by a Consensus string?
 - We can test a certain k-mer against each sequence to determine its best match and calculate the score of the set of matches
- The score of a motif can also be calculated row by row (i.e. sequence by sequence)

Motif finding

T	C	G	G	G	G	g	T	T	T	t	t	3
c	C	G	G	t	G	A	c	T	T	a	C	4
a	C	G	G	G	G	A	T	T	T	t	C	2
T	t	G	G	G	G	A	c	T	T	t	t	4
a	a	G	G	G	G	A	c	T	T	C	C	3
T	t	G	G	G	G	A	c	T	T	C	C	2
T	C	G	G	G	G	A	T	T	c	a	t	3
T	C	G	G	G	G	A	T	T	c	C	t	2
T	a	G	G	G	G	A	a	c	T	a	C	4
T	C	G	G	G	t	A	T	a	a	C	C	4
											<u>+ 3</u>	

$$3 + 4 + 0 + 0 + 1 + 1 + 1 + 5 + 2 + 3 + 6 + 4 = 30$$

T C G G G G A T T T C C

Motif Finding

- Number next to each row corresponds to the number of mismatches to the consensus string (Hamming distance)
- Given collection of k -mers $Motifs = \{Motif_1, \dots, Motif_t\}$ and k -mer *Pattern*:

$$d(Pattern, Motifs) = \sum_{i=1}^t \text{HAMMINGDISTANCE}(Pattern, Motifs_i)$$
$$\text{SCORE}(Motifs) = d(\text{CONSENSUS}(Motifs), Motifs)$$

- Is this new function useful for anything?

Motif Finding

Equivalent Motif Finding Problem:

Given a collection of strings, find a pattern and a collection of k -mers (one from each string) that minimizes the distance between all possible patterns and all possible collections of k -mers.

Input: A collection of strings Dna and an integer k .

Output: A k -mer *Pattern* and a collection of k -mers *Motifs*, one from each string in Dna , minimizing $d(Pattern, Motifs)$ among all possible choices of *Pattern* and *Motifs*.

Median String Problem

- It may seem like this problem is more difficult
- But for a given *Pattern*, we can easily find *Motifs*
- $\text{MOTIFS}(\text{Pattern}, \text{Dna})$ is a collection of k-mers that minimizes $d(\text{Pattern}, \text{Motifs})$ for all possible sets of *Motifs*
- If *Pattern* = AAA:
ttacctt**AAC**
g**ATA**tctgtc
ACGgcgttcg
ccct**AAA**gag
cgtc**AGA**ggt

Median String Problem

- Now we can consider each sequence separate instead of considering all Motif combinations

$$d(\text{Pattern}, \text{Text}) = \min_{\text{all } k\text{-mers Pattern' in Text}} \sum_{i=1}^t \text{HAMMINGDISTANCE}(\text{Pattern}, \text{Pattern}'_i)$$

- For example:

$$d(\text{GATTCTCA}, \text{gcaaaGACGCTGAccaa}) = 3$$

$$\text{MOTIF}(\text{GATTCTCA}, \text{gcaaaGACGCTGAccaa}) = \text{GACGCTGA}$$

- Note: there may be more than one k-mer with minimum distance

Median String Problem (Lab7 BA2B)

$$d(\textit{Pattern}, \textit{Dna}) = \sum_{i=1}^t d(\textit{Pattern}, \textit{Dna}_i)$$

Median String Problem:

Find a median string.

Input: A collection of strings *Dna* and an integer *k*.

Output: A *k*-mer *Pattern* minimizing $d(\textit{Pattern}, \textit{Dna})$ among all *k*-mers *Pattern*.

- Note: This requires two minimizations

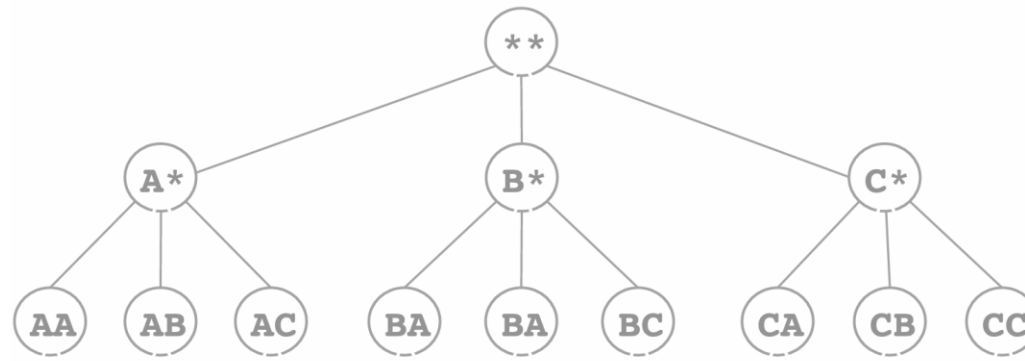
Median String Problem (Lab7 BA2B)

```
MEDIANSTRING(Dna, k)  
  distance  $\leftarrow \infty$   
  for each k-mer Pattern from AA . . . AA to TT . . . TT  
    if distance > d(Pattern, Dna)  
      distance  $\leftarrow d$ (Pattern, Dna)  
      Median  $\leftarrow$  Pattern  
  return Median
```

- Instead of $\mathcal{O}(n^t \cdot k \cdot t)$ now we have $\mathcal{O}(4^k \cdot n \cdot k \cdot t)$
- Still slow for larger *k* values (such as 15)

Branch-and-Bound

- Combinatorial optimization algorithms
- Uses an intelligent mechanism of lookahead to avoid the explicit enumeration of some of the solutions.
- The algorithm organizes the search space in a tree:
 - each leaf of the tree corresponds to a candidate solution
 - the internal nodes are partial sub-spaces

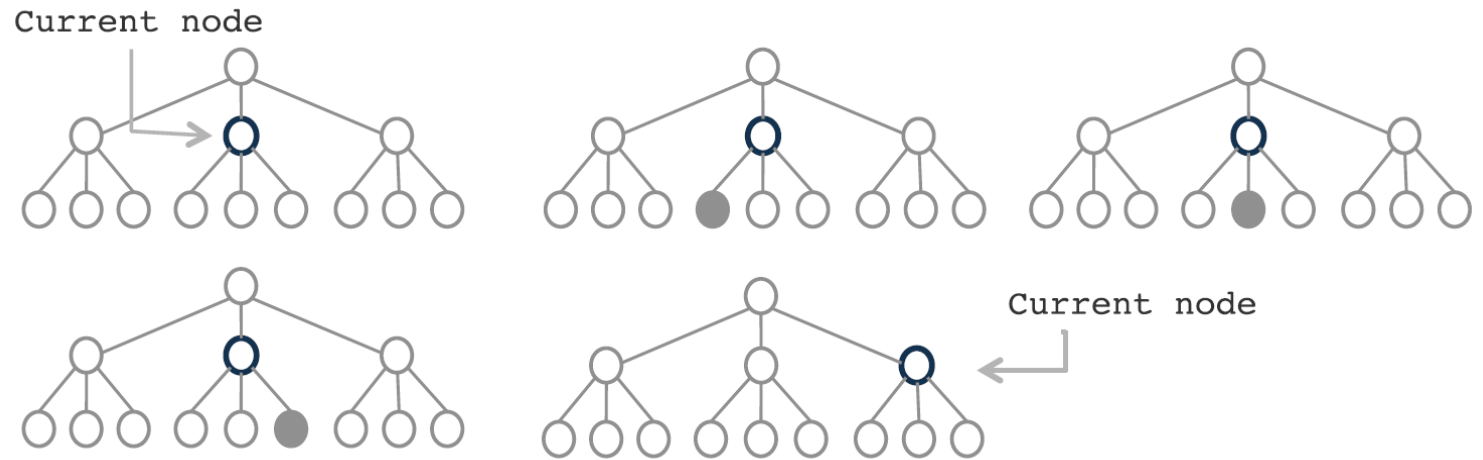


Branch-and-Bound

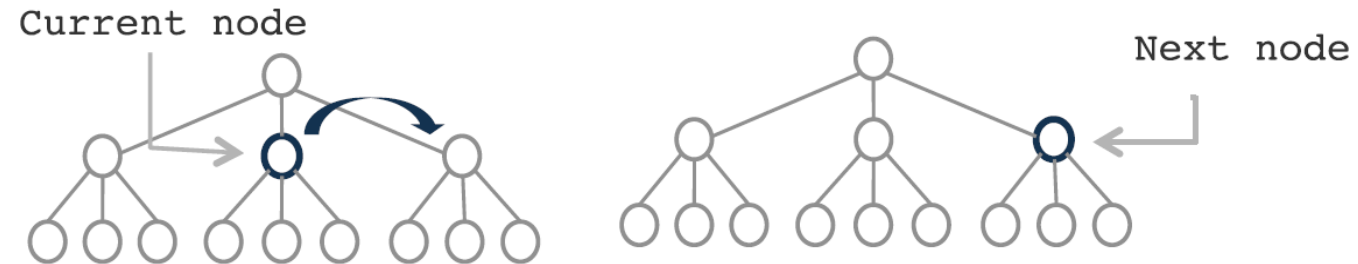
- At each internal node or branch, an upper or a lower bound of the objective function score for the respective sub-space can be calculated.
 - the algorithm will expand the search within that branch of the tree, if it has the potential to yield a better solution than the current one.
 - otherwise, all the candidates within that sub-space are discarded (not enumerated).

Branch-and-Bound

- Vertex enumeration



- Branch bypass



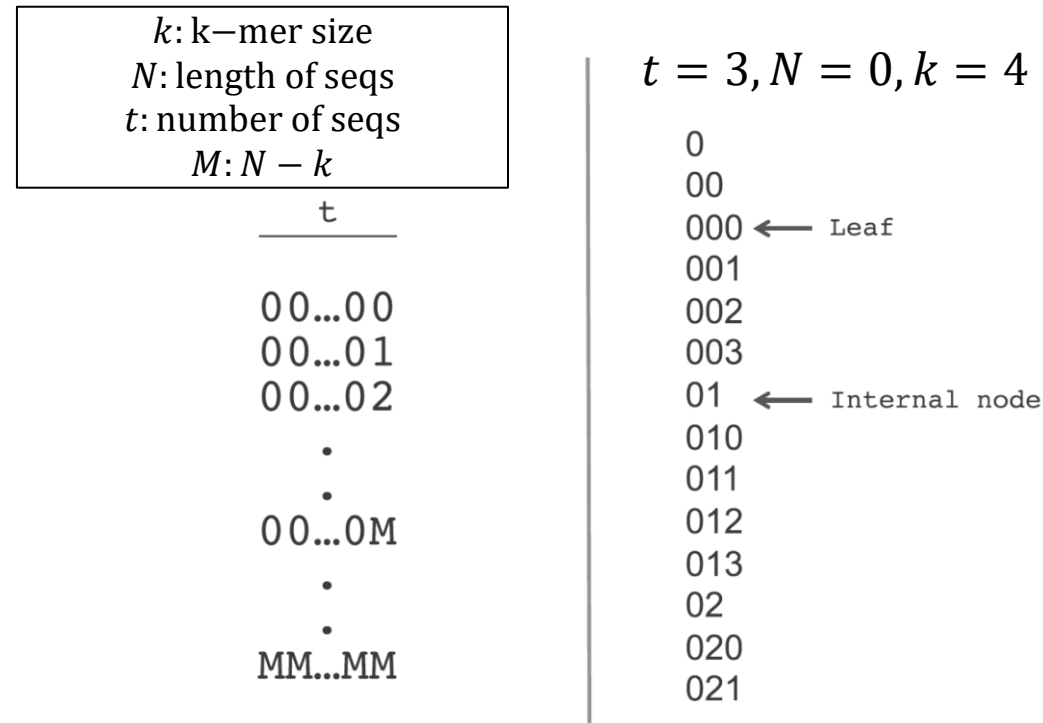
Branch-and-Bound

- To adapt the branch and bound algorithm for the motif discovery problem, we need to implement functionalities:
 - to traverse the tree
 - to calculate the bounds of the objective function values of the candidates.
- Objective function: We will consider a scoring function different to the previous one (higher is better/more conserved):

$$Score(M) = \sum_{i=1}^L \max_{k \in \Sigma} count(k, i)$$

Branch-and-Bound

- Representation of the enumeration space based on the vector of motif starting positions



Branch-and-Bound

- Estimate the bound for a sub-space, and test if it is worth to explore the solutions
 - Traversing the tree from the top to the bottom currently at level i .
 - For our t sequences the position of the selected k-mer can be divided as: $(s_1, \dots, s_i)$ and $(s_{i+1}, \dots, s_t)$
- Each sequence can contribute **at most with a score of k** .
- Fixing all the initial positions in the first i sequences, for the best-case scenario the **subsequent $t - i$ sequences can contribute with $(t - i) \times k$ for the current score**.
 - the bound for level i is, thus, given by
$$\text{Score}(s_1, \dots, s_i) + (t - i) \times k .$$
 - Bypass if this is lower than current best score

Heuristic Consensus

- Starts by considering only the first two input sequences to search for the initial positions s_1 and s_2 with the best score.
 - These provide the best partial contribution to the overall score.
- For the subsequent input sequences $i = 3, \dots, t$, the algorithm iterates through each sequence and selects the initial position that maximizes the score.
- Positions for the previous sequences $(s_1, \dots, s_{i-1})$ are fixed.

Heuristic Consensus

```
# class DeterministicMotifFinding:

    def heuristic_consensus(self):
        res = [0]*len(self.seqs)
        max_score = -1; partial = [0,0]
        for i in range(self.seq_size(0)-self.motif_size):
            for j in range(self.seq_size(1)-self.motif_size):
                partial[0] = i; partial[1] = j
                sc = self.score(partial);
                if (sc > max_score):
                    max_score = sc
                    res[0] = i; res[1] = j
        for k in range(2,len(self.seqs)):
            partial = [0]*(k+1)
            for j in range(k):
                partial[j] = res[j]
            max_score = -1
            for i in range(self.seq_size(k)-self.motif_size):
                partial[k] = i
                sc = self.score(partial)
                if (sc > max_score):
                    max_score = sc; res[k] = i
        return res
```

Heuristic Consensus

```
def test_heuristic_consensus():  
    seq1 = MySeq("ACATATCAT")  
    seq2 = MySeq("AGATATTAG")  
    seq3 = MySeq("AACAGATCT")  
    seq4 = MySeq("GCATCGATT")  
    mf = DeterministicMotifFinding(3, [seq1, seq2, seq3, seq4])  
    best_motif = mf.heuristic_consensus()  
    print ("Solution: ", best_motif)  
    print ("Score: ", mf.score(best_motif))
```

```
In : test_heuristic_consensus()  
Solution:  [2, 2, 1, 2]  
Score:  10
```

- Highly dependent on the first two initial positions
 - which on its turn depend on the order of the input sequences

Summary

- Importance of Motif Finding
- Scoring Motifs
- Brute force search
- Median String approach
- Branch-and-bound
- Heuristic Search

Bibliography

- Compeau Chapter 2
- Rocha Chapter 10